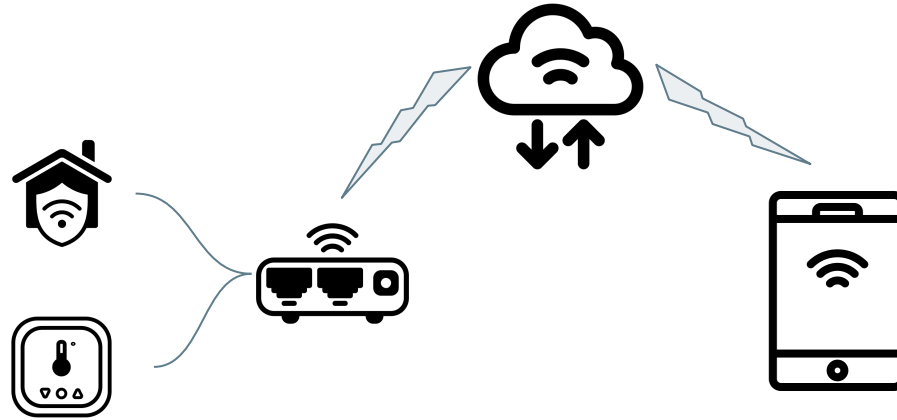


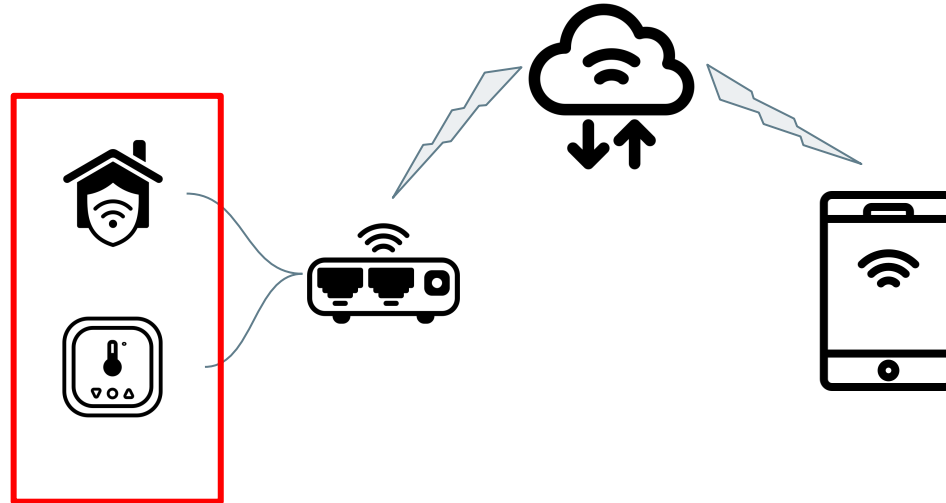
Internals

Hardware and Software

IoT System

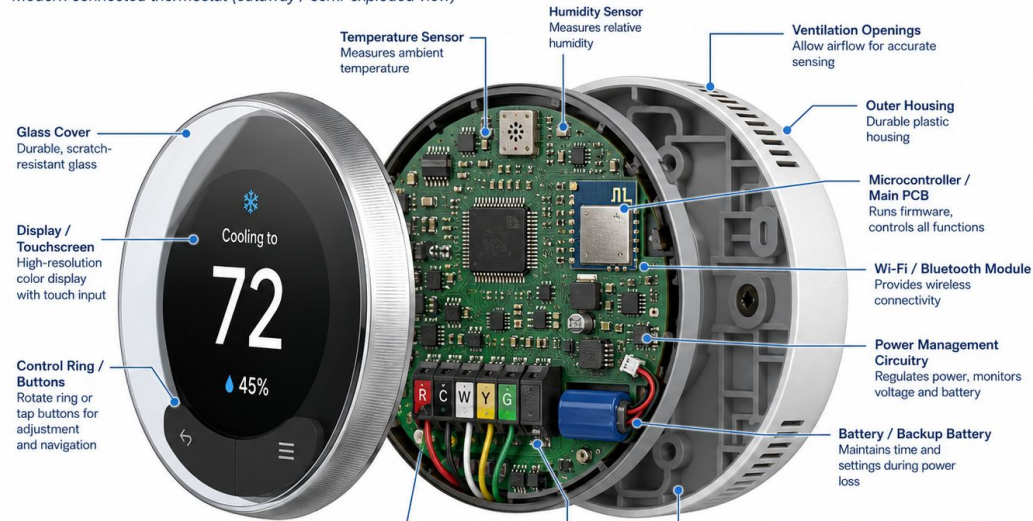


IoT Device

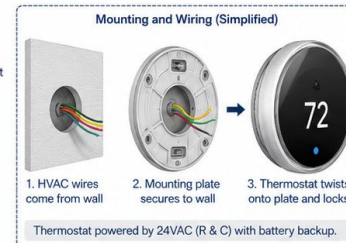


IoT Device: Hardware

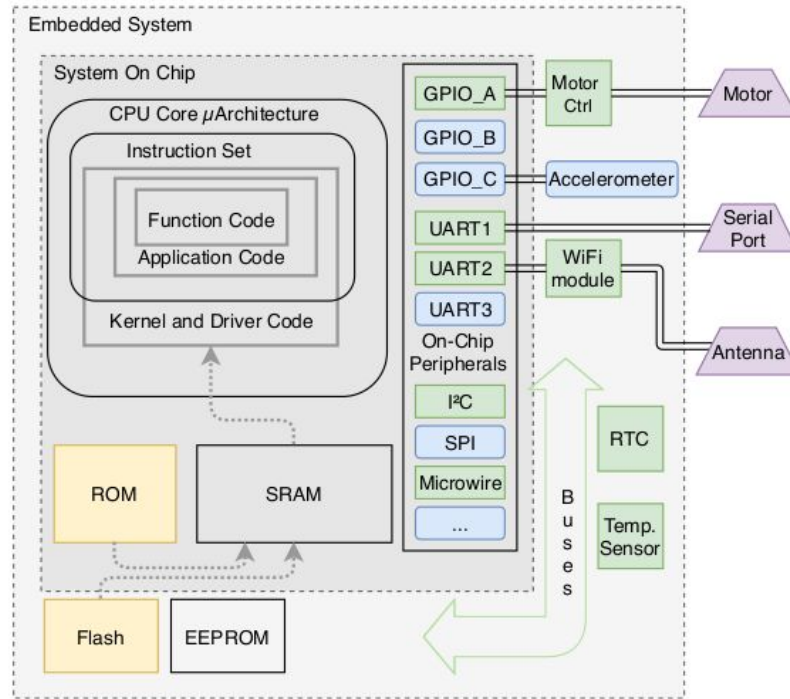
Modern connected thermostat (cutaway / semi-exploded view)



HVAC Terminal Functions	
R	24VAC Power (from system)
C	24VAC Common
W	Heat
Y	Cool
G	Fan



IoT Device: Hardware



IoT Device: Hardware

- Sensors/Actuators.
- Connectivity.
- Application Processor:
 - Usually a RISC processor (why?):
 - Microcontroller Unit (MCU), e.g., ESP8266.
 - ARM based processors.

MCU v/s Traditional Processor (e.g., ARM)

MCU (usually) does not have:

- Privilege rings (e.g., EL0, EL1, etc).
- Memory management unit.
- High number of General Purpose Registers.

MCUs have:

- Low power consumption.
- Simpler processor management.



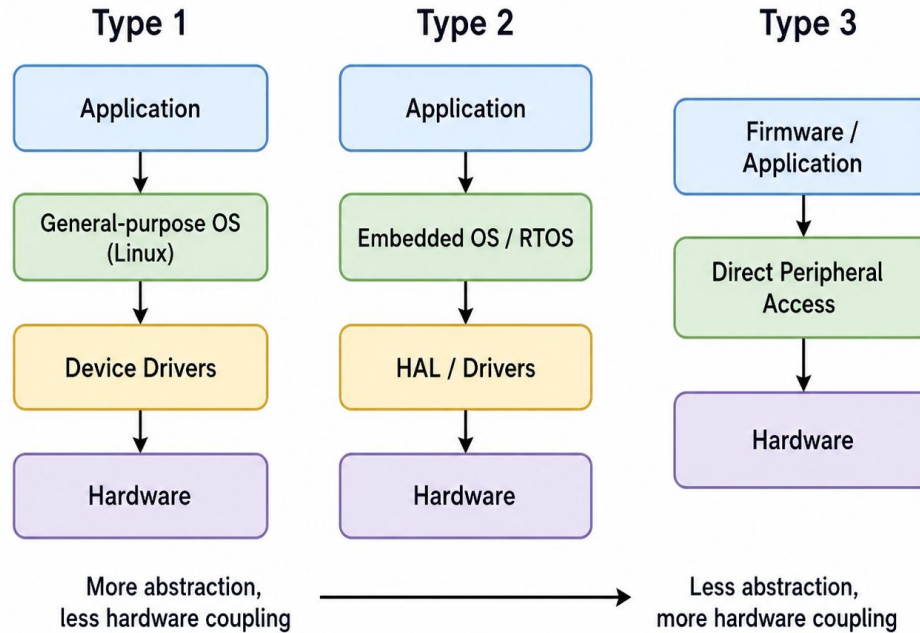
IoT Device: Software

What runs inside IoT Device?

(Most) Traditional Systems:

- Applications <-> OS <-> Drivers <-> Hardware

IoT Device: Common Software Architectures



IoT Device: Type 1

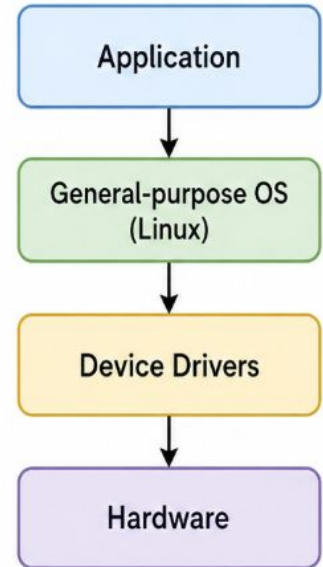
OS usually customized for embedded devices, e.g., Open WRT, Embedded Linux.

Web-server (e.g., nginx) is the main application that interacts with external world.

Common devices: Routers, Smart TVs,

<https://www.shodan.io/search/facet?query=Tenda&facet=os>

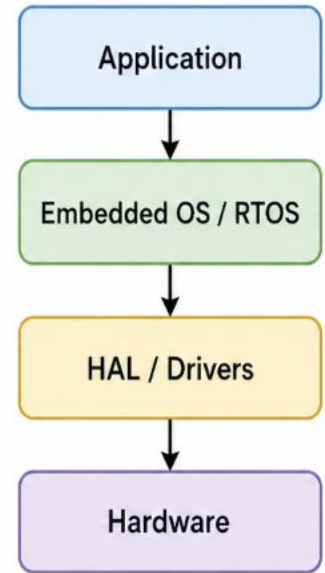
Type 1



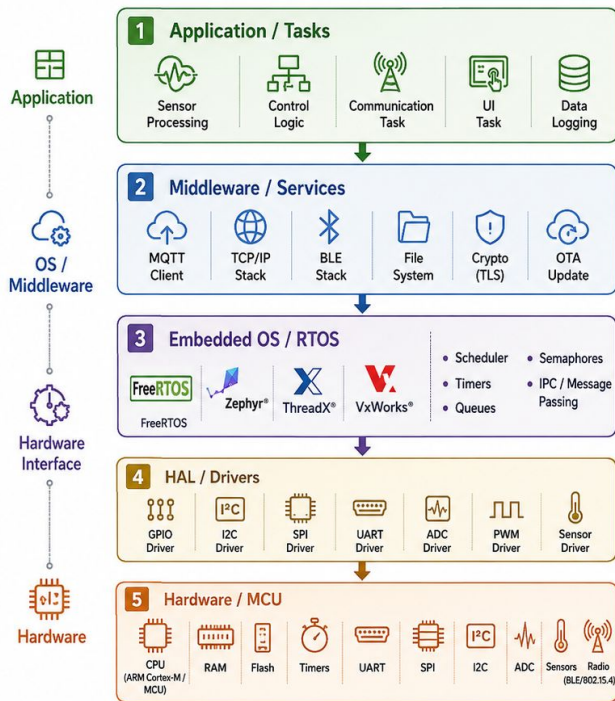
IoT Device: Type 2

- Uses specialized **simple** Real time operating systems (RTOS): No standard OS abstractions.
- Task based abstraction.
- **Single Address Space: All tasks have access to each other.**
- Libraries try to provide advanced services.
- https://github.com/MayaPosch/FreeRTOS_HTTP_server/blob/master/Core/Src/main.c

Type 2



IoT Device: Type 2



Type 2 Key Characteristics

- ✓ Some OS abstractions
- ✓ More tightly coupled to hardware than Type 1
- ✓ Real-time scheduling
- ✓ Lower resource usage than Type 1
- ✓ Common in MCUs and control devices

Type 2 IoT Device Examples



Industrial controller / PLC

Cortex-M + RTOS



Smart meter

MCU + RTOS



Wearable / fitness band

Cortex-M + RTOS



BLE sensor node

Cortex-M + RTOS



Automotive ECU

Cortex-M + RTOS



Smart thermostat controller

MCU + RTOS



Deterministic / Real-time
Predictable timing and bounded latency



Moderate abstraction
Provides OS services and drivers while staying close to hardware



MCU / Cortex-M class
Designed for resource-constrained microcontrollers and SoCs



Common RTOS
FreeRTOS, Zephyr, ThreadX

IoT Device: Type 1 v/s Type 2

- Complexity of OS?
- Security/Isolation?
- Code size?
- Processor support?
- Programming complexity?

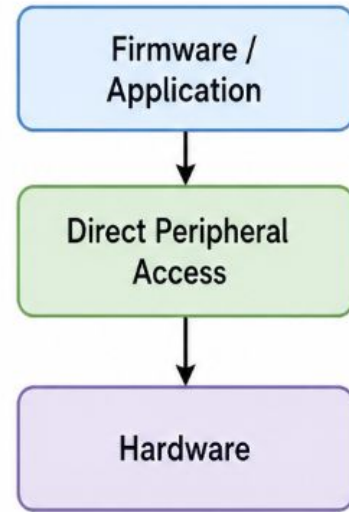
IoT Device: Type 3 (Bare-Metal or Monolithic systems)

Complete control of peripherals.

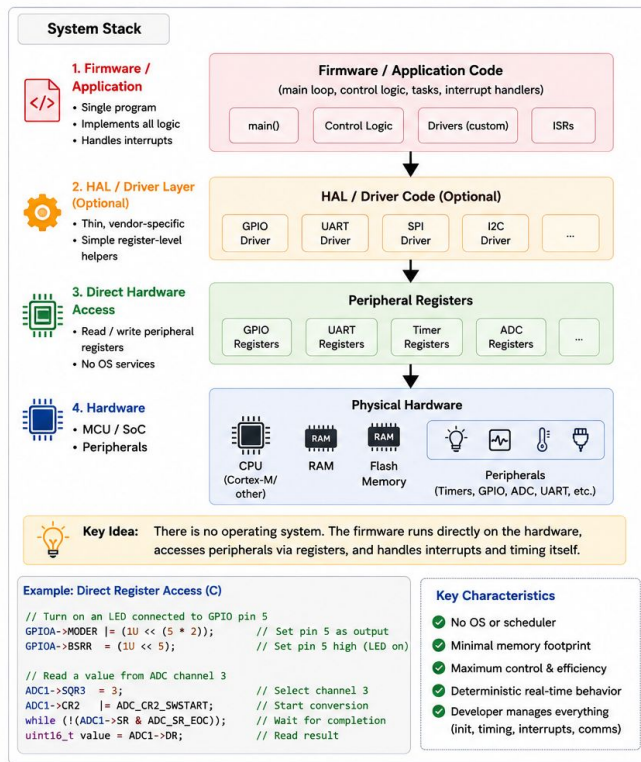
(Usually) fixed tasks.

- Initialize peripherals.
- Continuously listen for commands and perform action on peripherals.
- Read peripheral state and update.
- https://github.com/dekuNukem/STM32_tutorials/blob/master/lesson0_intro_blinkLED/sample_code/Src/main.c

Type 3



IoT Device: Type 3



Examples of Type 3 (Bare-Metal) Devices

Temperature Sensor Node



- Reads temperature
- Sleeps most of the time
- Sends data via UART / radio

Smart Light Bulb Controller



- Controls LED brightness
- Handles button input
- No OS, minimal firmware

Motor Controller (ESC)



- Reads PWM input
- Drives motor phases
- Real-time control loops

Remote Key Fob



- Reads button presses
- Sends RF signal
- Very low-power firmware

Digital Watch



- Keeps time
- Drives LCD segments
- No OS

Industrial Sensor (Pressure, Flow, etc.)



- Samples sensor
- Processes data
- Outputs via I2C / 4-20mA



Typical Hardware

MCUs: ARM Cortex-M0/M0+/M3/M4/M33, AVR, MSP430, RISC-V, etc.

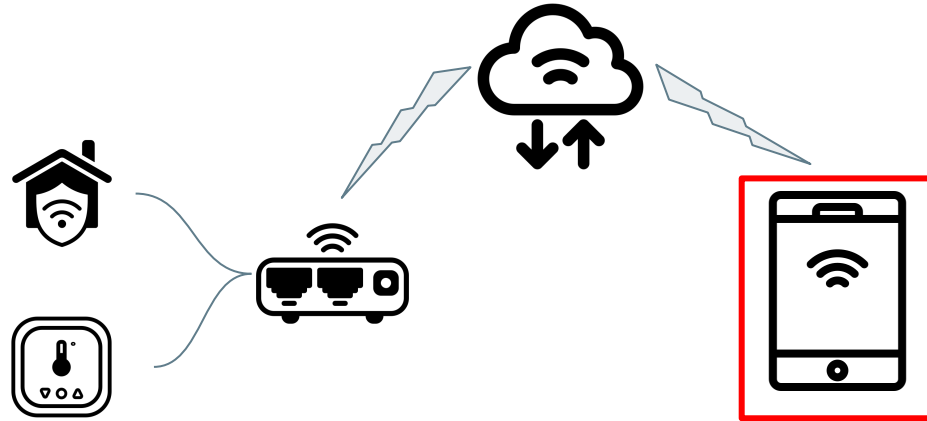


Memory: Tens of KB RAM, Tens to Hundreds of KB Flash



Peripherals: Timers, GPIO, ADC, UART, SPI, I2C, PWM, etc.

IoT System: Controller



Controller

Goals:

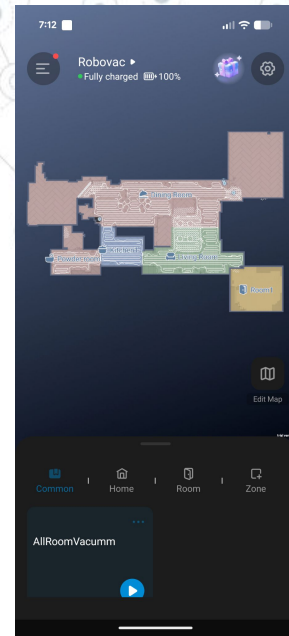
- Easy to use.
- Secure against host environment.

Common controller type?

Controller

Goals:

- Easy to use.
- Secure against host environment.



Common controller type? Smartphone App (Companion App)

Controller: Smartphone App

- Design Principles (Modularity):
 - Separate UI from Data model.
 - Separate Data model from device interaction.
 - Separate device interaction from connectivity.
- Should be easy to adopt for new connectivity method.
- Should be easy to add a new IoT device.
- Should be easy to port for different Smartphones.
- Always use underlying framework abstractions, e.g., Android libraries, iOS SDK, etc.

Controller: Smartphone App

1. Software Components



User Interface (UI)

- Activities / Fragments / Compose UI
- Displays device status and data
- Takes user input and shows feedback



ViewModel / Logic

- Holds UI state
- Handles user actions
- Performs validation and business logic



Repository

- Single source of truth for app data
- Coordinates data from different sources
- Exposes simple APIs to ViewModel



Communication Layer

- Handles connectivity with devices or cloud
- BLE / Wi-Fi / MQTT / HTTP
- Sends commands and receives data



Local Storage

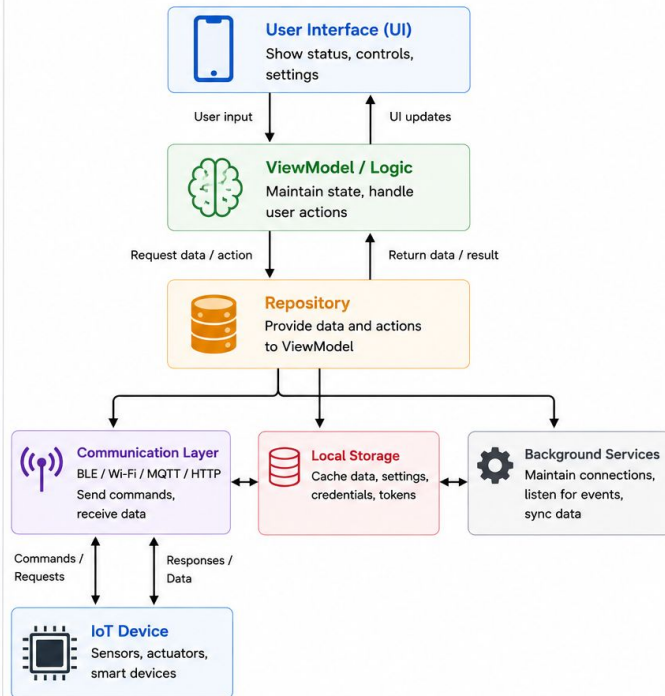
- Stores device info, user preferences
- Credentials, tokens, cached data
- Room Database / SharedPreferences



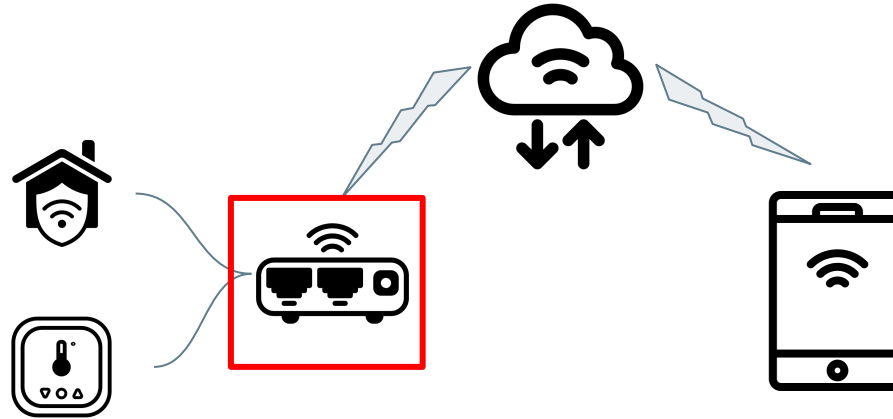
Background Services

- Runs long-running tasks in background
- Maintains connections / listens for events
- Syncs data and shows notifications

2. How Components Interact



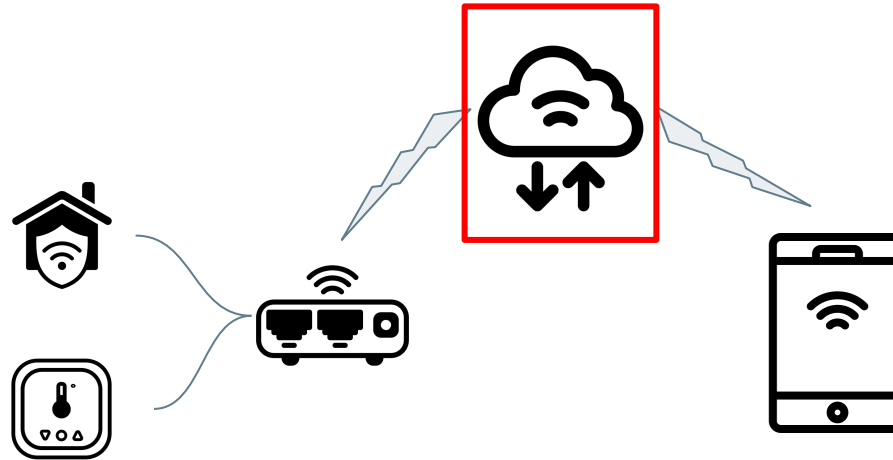
IoT System: Edge Device



Edge/Hub Device

- Full featured system.
- Multi-connectivity support, i.e., WIFI, BLE, ZigBee.
- Runs full features OS with some special application layer (OpenHome SDK, Matter SDK, etc).
 - E.g., Amazon Echo, Samsung Smarthings

IoT System: Cloud



IoT System: Cloud

- Self-hosted or hosted by existing cloud platforms (e.g., Google Cloud, Amazon AWS, Microsoft Azure, etc).
- Manage and control all IoT devices of the vendor.
- Usually has a database, scaling infrastructure, and admin panel.

IoT System: Cloud SDKs

- Cloud providers have SDKs that enable easy management of devices:
 - AWS IoT SDK
 - Azure IoT SDK
 - Arduino IoT Cloud
- Each SDK has device side component and cloud side component.

AWS IoT SDK: Device side

```
from awscrt import mqtt
from awsiot import mqtt_connection_builder
import json

# Connect device to AWS IoT Core
connection = mqtt_connection_builder.mtls_from_path(
    endpoint="xxxxxx-ats.iot.us-east-1.amazonaws.com",
    cert_filepath="device.pem.crt",
    pri_key_filepath="private.pem.key",
    ca_filepath="AmazonRootCA1.pem",
    client_id="temperature-sensor-17"
)

connection.connect().result()

# Sensor reading
temperature = 23.5

# Publish telemetry
connection.publish(
    topic="devices/sensor17/temperature",
    payload=json.dumps({"temperature": temperature}),
    qos=mqtt.QoS.AT_LEAST_ONCE
)

print("Temperature sent")
```

AWS IoT SDK: Cloud side (AWS IoT Rule)

```
SELECT
    temperature,
    timestamp,
    topic(2) AS deviceId
FROM
    'devices/+/temperature'
```

AWS IoT SDK: Cloud side (AWS IoT Rule -> Lambda function)

```
SELECT
  temperature,
  timestamp,
  topic(2) AS deviceId
FROM
  'devices/+/temperature'
```

AWS IoT > Message routing > Rules > Create rule

Step 1
[Specify rule properties](#)

Step 2
[Configure SQL statement](#)

Step 3
Attach rule actions

Step 4
Review and create

Attach rule actions [Info](#)

An action routes data to a specific AWS service.

SQL statement [Back](#)

```
SELECT
  serialNumber as serialNumber,
  clickType as clickType,
FROM 'device/+/data'
```

Rule actions

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional activities that occur when messages arrive, like storing them in a database, invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

▼

Lambda

Send a message to a Lambda function

▼

Remove

Lambda function [Info](#)

iotcore-us-east-1-dev-lambda-iotActions

↺

View [🔗](#)

Create a Lambda function [🔗](#)

Lambda function version

\$LATEST

↺

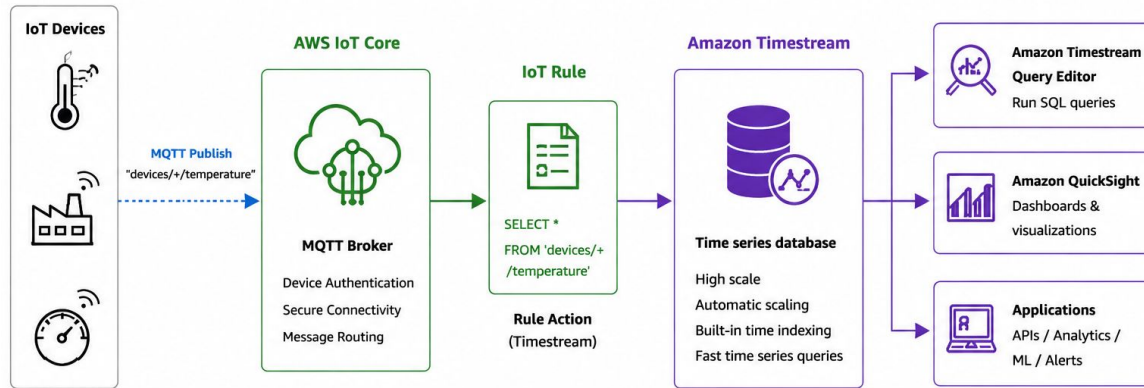
AWS IoT SDK: Cloud side (lambda handler)

```
def lambda_handler(event, context):  
  
    device = event["deviceId"]  
    temperature = event["temperature"]  
  
    print(device, temperature)  
  
    if temperature > 30:  
        print("High temperature alert")
```



Amazon
TimeStream

AWS IoT SDK: Typical Architecture



Example IoT Message

```
{
  "deviceId": "sensor17",
  "temperature": 23.5,
  "humidity": 45.2,
  "ts": 1711027200000
}
```

Data in Amazon Timestream (sample table: sensor_data)

time (timestamp)	deviceId	temperature (double)	humidity (double)
2024-03-21 10:00:00.000	sensor17	23.5	45.2
2024-03-21 10:01:00.000	sensor17	23.6	45.0
2024-03-21 10:02:00.000	sensor17	23.7	44.8
2024-03-21 10:03:00.000	sensor17	23.6	44.6
...	...	...	...

Time series visualization (example)

